ME403 – Introduction to Robotics

Lab 01: Forward Kinematics

Sabanci University | Spring 2025-26

Prepared by Yunus Emre Danabas for ME403

Lab Outcomes

By the end of this lab, you will be able to:

- Explain the difference between body-frame and absolute joint angles.
- Write Python calculation scripts that command the robot without a GUI.
- Apply the planar 2R forward kinematics (FK) formula to predict end-effector position.
- Execute multi-step joint-space trajectories using a single function call per step.
- Compare FK predictions to the robot's reported Cartesian pose and interpret the error.

1. Body-Frame Joint Angles

1.1. What "body-frame" means

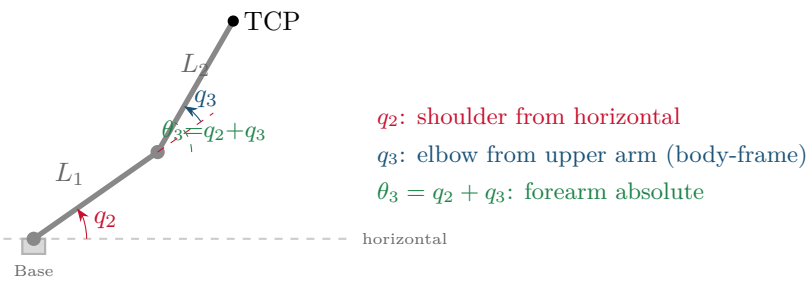
A joint angle can be measured in two different ways:

- **Absolute (world-frame):** the angle from a fixed world reference (e.g., from horizontal). The firmware internally uses this convention.
- **Body-frame (relative):** the angle from the previous link's own direction. This is the physically intuitive measurement — you can check it with a protractor placed on the robot.

In this lab you always work with **body-frame angles** $q = [q_1, q_2, q_3, q_4]$. The utility functions convert to firmware values automatically.

1.2. Diagram

The diagram below shows the key difference. q_2 is the shoulder elevation from horizontal (same in both conventions). q_3 is the elbow angle *from the upper-arm direction* (body-frame) — **not** from horizontal. The absolute elbow angle $\theta_3 = q_2 + q_3$.



1.3. Body-Frame to Firmware Conversion

The two robots handle the conversion differently because of their internal mechanics. `moveMagician()` and `moveMG400()` perform this conversion for you.

Joint	Body-frame input	Magician firmware	MG400 firmware
J1	q_1	q_1	q_1
J2	q_2	q_2	q_2
J3	q_3	q_3 (quirk: Magician J3 IS body-frame)	$q_2 + q_3$
J4	q_4	$q_2 + q_3 + q_4$	$q_2 + q_3 + q_4$

DH Convention (coming in Week 4)

Professional robot software uses the **Denavit-Hartenberg (DH)** convention to systematically assign coordinate frames to each joint. DH parameters give you the same geometric insight as body-frame angles but with a standardized notation that works for any kinematic chain. You will study DH in Week 4. For now, body-frame angles are the physically intuitive equivalent.

2. Forward Kinematics Formula

Given body-frame joint angles $q = [q_1, q_2, q_3, q_4]$, the end-effector Cartesian position is predicted by the **planar 2R model**:

$$\theta_2 = q_2 \quad \text{(shoulder absolute angle from horizontal)} \tag{1}$$
$$\theta_3 = q_2 + q_3 \quad \text{(forearm absolute angle from horizontal)} \tag{2}$$

$$r_{\text{reach}} = L_1 \cos\theta_2 + L_2 \cos\theta_3 \tag{3}$$
$$Z_{\text{tip}} = Z_{\text{base}} + L_1 \sin\theta_2 + L_2 \sin\theta_3 \tag{4}$$

$$X = r_{\text{reach}} \cos q_1 \tag{5}$$
$$Y = r_{\text{reach}} \sin q_1 \tag{6}$$

where Z_{base} is the height of the shoulder (J2) pivot above the mounting surface. Equations (3)–(6) apply in Python as:

```
import math
theta2 = math.radians(q[1])
theta3 = math.radians(q[1] + q[2])    # forearm absolute angle
reach  = L1*math.cos(theta2) + L2*math.cos(theta3)
Z_pred = Z_base + L1*math.sin(theta2) + L2*math.sin(theta3)
X_pred = reach * math.cos(math.radians(q[0]))
Y_pred = reach * math.sin(math.radians(q[0]))
```

Robot	L_1 (mm)	L_2 (mm)	Z_{base} (mm, nominal)	Notes
Dobot Magician	135	147	103	Verify: move to $q = [0, 0, 0, 0]$, read Z
DOBOT MG400	175	175	116	Verify: move to $q = [0, 0, 0, 0]$, read Z

Singularity Warning

When $q_2 \approx 0$ and $q_3 \approx 0$, the arm is near-horizontal and the Jacobian is degenerate: small joint errors produce large end-effector errors. Avoid using these configurations for FK verification tasks. Keep $q_2 \in [10°, 60°]$ and $q_3 \in [0°, 40°]$ for reliable results.

3. API Reference

Magician (`import utils as U`)

Call	Description
`bot = U.setup(port=None)`	Connect, clear alarms, go home. Returns the `bot` object.
`U.moveMagician(bot, q)`	Move to body-frame $q = [q_1, q_2, q_3, q_4]$ (degrees). Prints conversion.
`q = U.get_joints(bot)`	Returns body-frame (q_1, q_2, q_3, q_4) in degrees.
`x,y,z,r = U.get_pose(bot)`	Returns Cartesian pose (mm / degrees).
`U.teardown(bot)`	Go home and close serial port. Call in `finally` block.
`U.L1`, `U.L2`, `U.Z_base`	Link lengths and shoulder height constants (mm).

MG400 (`import utils_mg400 as U`)

Call	Description
`db, mv = U.setup(robot=1)`	Connect, enable, clear errors, go home. Returns `(dashboard, move_api)`.
`U.moveMG400(mv, q)`	Move to body-frame $q = [q_1, q_2, q_3, q_4]$. Prints conversion.
`q = U.get_joints(db)`	Returns body-frame (q_1, q_2, q_3, q_4) in degrees.
`x,y,z,r = U.get_pose(db)`	Returns Cartesian pose.
`U.teardown(db, mv)`	Go home, disable robot, close TCP. Call in `finally` block.
`U.L1`, `U.L2`, `U.Z_base`	Link lengths and shoulder height constants (mm).

Example script (Magician):

```
import utils as U
import math

bot = U.setup()                    # connects and homes
try:
    L1, L2, Z_base = U.L1, U.L2, U.Z_base

    q = [0, 30, 20, 0]             # body-frame angles (degrees)
    U.moveMagician(bot, q)
    x, y, z, r = U.get_pose(bot)
    print(f"X={x:.1f}  Y={y:.1f}  Z={z:.1f}")

    # Predict using FK formula
    th2 = math.radians(q[1])
    th3 = math.radians(q[1] + q[2])
```

```
    Z_pred = Z_base + L1*math.sin(th2) + L2*math.sin(th3)
    print(f"Z_pred={Z_pred:.1f}  Z_actual={z:.1f}  error={abs(z-Z_pred):.1f
        } mm")
finally:
    U.teardown(bot)
```

4. Lab Tasks

Pre-Lab Worksheet (complete before the lab session)

Answer these on paper before touching the robot:

1. For $q = [0, 40, 20, 0]$, compute θ_3 and the predicted (X, Z) for the Magician (L_1=135, L_2=147, Z_{base}=103).
2. For $q = [30, 30, 15, 0]$, compute the predicted (X, Y, Z) for the MG400 (L_1=175, L_2=175, Z_{base}=116). Note the effect of $q_1 = 30°$.
3. Choose a configuration where the arm is fully extended ($q_3 = 0$). What does the FK formula reduce to?

Task 0 — Measure Z_{base} (5 min)

Goal: Verify the nominal Z_{base} value for your robot.

1. `setup()` homes the arm (all joints to 0).
2. Call `get_pose()` and record the reported Z.
3. Compare to the nominal value in `utils.py`.
4. If the measured Z differs by more than 10 mm, update `Z_base` in your script.

Why: The FK formula uses Z_{base} as the shoulder height offset. An incorrect value shifts all your Z predictions by a fixed amount.

Task 1 — Single Configuration (15 min)

Goal: Hand-calculate the FK prediction, then verify with the robot.

1. Choose body-frame angles $q = [q_1, q_2, q_3, q_4]$ (keep $q_2 \in [10°, 60°]$, $q_3 \in [0°, 40°]$).
2. *On paper:* compute X_{pred} and Z_{pred} using the FK formula.
3. Fill in `q1 = [...]` in the template and run the script.
4. Print your prediction and the robot's actual pose.
5. Report the XY error and Z error.

Deliverable: Your chosen q, the hand-calculated prediction, and the comparison printout.

Task 2 — Multi-Step Trajectory (15 min)

Goal: Design and execute a trajectory through at least 3 configurations.

1. Fill in the `configurations` list with 3–5 rows.
2. Choose configurations that form a visible end-effector path (e.g., the arm sweeps upward, then rotates at the base, then returns).
3. Run the script and record each actual pose.
4. *Qualitatively* describe the path the end-effector traces.

Deliverable: The `configurations` list and the trajectory printout with actual poses.

Task 3 — FK Verification (20 min)

Goal: Implement `fk_predict()` and compare predicted vs. actual pose.

1. Fill in the three `???` expressions in `fk_predict()`.
2. Uncomment the comparison loop to print the error table.
3. Report the **XY error** $\sqrt{(X_p - X_a)^2 + (Y_p - Y_a)^2}$ and **Z error** $|Z_p - Z_a|$ for each configuration.
4. Answer in your lab report: which error axis is larger? Why?

Expected result: XY error $\approx$ 5–20 mm (linkage approximation); Z error $\approx$ 5–20 mm (base height tolerance). If Z error is > 50 mm, recheck your Z_{base} measurement.

Deliverable: The completed `fk_predict()` function, the error table, and 2–3 sentences explaining the dominant error source.

Task 4 (Optional) — Inverse Kinematics Preview

Goal: Given a target Cartesian position, find the joint angles analytically.

For a target (X_t, Y_t, Z_t) in the plane $q_1 = 0$:

$$
\begin{aligned}
r_t &= \sqrt{X_t^2} = X_t \quad (q_1 = 0) \\
z_t &= Z_t - Z_{\text{base}} \\
\cos q_3 &= \frac{r_t^2 + z_t^2 - L_1^2 - L_2^2}{2L_1L_2} \quad \Rightarrow \quad q_3 = \arccos(\cdot) \\
q_2 &= \arctan 2(z_t, r_t) - \arctan 2(L_2 \sin q_3, L_1 + L_2 \cos q_3)
\end{aligned}
$$

Verify: call `moveMagician(bot, [0, q2, q3, 0])` and check that the robot reaches (X_t, Z_t).

5. Setup

Magician — Environment Setup

Run from: `Students/01_SecondWeek/`

Linux:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r magician/requirements.txt
cd magician
python lab01_fk.py
```

Windows (PowerShell):

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r magician\requirements.txt
cd magician
python lab01_fk.py
```

MG400 — Environment Setup

Step 1 — Network: Static IP `192.168.2.100/24`. Verify: `ping 192.168.2.7`.

Step 2 — SDK (one-time, from `dobot_ws/` root):

```
git clone https://github.com/Dobot-Arm/TCP-IP-4Axis-Python.git vendor/
    TCP-IP-4Axis-Python
```

Step 3 — Linux:

```
python3 -m venv .venv && source .venv/bin/activate
pip install -r mg400/requirements.txt
cd mg400 && python lab01_fk.py
```

Windows:

```
python -m venv .venv && .\.venv\Scripts\Activate.ps1
pip install -r mg400\requirements.txt
cd mg400 && python lab01_fk.py
```

6. Submission Checklist

Submission Checklist

`lab01_fk.py` runs from start to finish without errors (no `None` values).
Task 0: Z_{base} measured and reported; script uses the measured value.
Task 1: q is filled in (not `None`); hand-calculation shown and compared to actual.
Task 2: At least 3 configurations defined; trajectory output printed.
Task 3: `fk_predict()` implemented (no `???`); error table printed for each step.
Error decomposition: XY error and Z error reported separately.
Lab report paragraph (3–5 sentences): explain why the simplified FK model produces the observed errors. Identify the dominant source.
Robot returns to home position after teardown without collision.

7. Safety Reminders

Mandatory Safety Rules

1. **Workspace:** Maintain a clear area (40 cm radius for Magician, 50 cm for MG400).
2. **Joint ranges:** Use $q_2 \in [10°, 60°]$ and $q_3 \in [0°, 40°]$ for initial experiments. `moveMagician()` / `moveMG400()` clamp with a warning — check the output.
3. **Teardown:** Always include `teardown()` in a `finally` block. Never leave the robot enabled and unattended.
4. **MG400 Z-axis:** Z cannot go negative. Keep $Z > 5$ mm at all times.
5. **Port conflict (Magician):** Close DobotStudio before running any Python script.